Design, Implement and Validate an Object-Oriented Model for a Campus Smart-Parking System

Name: Karnati Nikhil
Register Number: 192525307
Course: Programming in Java
Course code: CSA0922

GitHub Repository: https://github.com/karnatinikhil-sudo/CSA0922-Java-Programming

TASK 1: Identify Core Entities and Design Classes

Aim

To identify the core entities of a campus smart-parking system and design suitable Java classes using object-oriented programming concepts.

Core Entities

1. Vehicle: Represents a vehicle entering the parking facility.
2. ParkingSlot: Represents an individual parking slot.
3. ParkingFloor: Represents a floor containing multiple parking slots.
4. ParkingTicket: Stores vehicle entry time and parking details.
5. ParkingFacility: Manages parking floors, slots, and vehicle allocation.

Java Code

```
class Vehicle {
    String vehicleNumber;
    String vehicleType;

    Vehicle(String number, String type) {
        vehicleNumber = number;
        vehicleType = type;
    }

    void display() {
        System.out.println("Vehicle Number: " + vehicleNumber);
        System.out.println("Vehicle Type: " + vehicleType);
    }
}
```

```
class ParkingSlot {
    int slotNumber;
    boolean occupied;

    ParkingSlot(int number) {
        slotNumber = number;
        occupied = false;
    }

    void display() {
        System.out.println("Slot Number: " + slotNumber);
        System.out.println("Occupied: " + occupied);
    }
}

public class Main {
    public static void main(String[] args) {
        Vehicle v = new Vehicle("TN01AB1234", "Car");
        ParkingSlot s = new ParkingSlot(1);

        v.display();
        s.display();
    }
}
```

Output

Main.java

Run ▶

```
    boolean occupied;

    ParkingSlot(int number) {
        slotNumber = number;
        occupied = false;
    }

    void display() {
        System.out.println("Slot Number: " + slotNumber);
        System.out.println("Occupied: " + occupied);
    }
}

public class Main {
    public static void main(String[] args) {
        Vehicle v = new Vehicle("TN01AB1234", "Car");
        ParkingSlot s = new ParkingSlot(1);

        v.display();
        s.display();
    }
}
```

Output

```
Vehicle Number: TN01AB123
Vehicle Type: Car
Slot Number: 1
Occupied: false
```

Conclusion

The core classes represent the main entities of the smart-parking system and provide a foundation for further development.

TASK 2: Apply Encapsulation

Aim

To implement encapsulation by keeping data members private and providing public methods to access and modify them.

Java Code

```
class ParkingSlot {
    private int slotNumber;
    private boolean occupied;

    ParkingSlot(int number) {
        slotNumber = number;
        occupied = false;
    }

    public int getSlotNumber() {
        return slotNumber;
    }

    public boolean isOccupied() {
        return occupied;
    }

    public void setOccupied(boolean status) {
        occupied = status;
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        ParkingSlot slot = new ParkingSlot(101);

        System.out.println(slot.getSlotNumber());
        System.out.println(slot.isOccupied());

        slot.setOccupied(true);
        System.out.println(slot.isOccupied());
    }
}
```

Output



Explanation

Encapsulation protects the internal data of a class. Private variables can be accessed through public getter and setter methods.

Conclusion

Encapsulation improves data security, control, and maintainability.

TASK 3: Implement Abstraction Using an Abstract Class

Aim

To demonstrate abstraction by defining common parking operations in an abstract class and implementing them in a subclass.

Java Code

```
abstract class ParkingService {
    abstract void parkVehicle();

    void displayMessage() {
        System.out.println("Welcome to Smart Parking");
    }
}

class CarParking extends ParkingService {
    @Override
    void parkVehicle() {
        System.out.println("Car parked successfully");
    }
}

public class Main {
    public static void main(String[] args) {
        CarParking car = new CarParking();

        car.displayMessage();
        car.parkVehicle();
    }
```

}

Output



Explanation

An abstract class can contain abstract and concrete methods. The subclass implements the abstract method.

Conclusion

Abstraction hides implementation details and provides a clear structure for parking operations.

TASK 4: Demonstrate Inheritance

Aim

To use inheritance to represent different types of vehicles in the campus smart-parking system.

Java Code

```
class Vehicle {
    String number;

    Vehicle(String number) {
        this.number = number;
    }

    void display() {
        System.out.println("Vehicle Number: " + number);
    }
}

class Car extends Vehicle {
    Car(String number) {
        super(number);
    }

    void park() {
        System.out.println("Car parked");
    }
}

public class Main {
```

```
    public static void main(String[] args) {
        Car c = new Car("TN01AB1234");


        c.display();
        c.park();
    }
}
```

Output



Explanation

The Car class inherits the properties and methods of the Vehicle class using the extends keyword. The super() statement calls the parent class constructor.

Conclusion

Inheritance promotes code reuse and simplifies the implementation of different vehicle types.

TASK 5: Demonstrate Polymorphism

Aim

To demonstrate runtime polymorphism by overriding a method in a subclass.

Java Code

```
class Vehicle {
    void park() {
        System.out.println("Vehicle is parked");
    }
}

class Car extends Vehicle {
    @Override
    void park() {
        System.out.println("Car is parked in a car slot");
    }
}

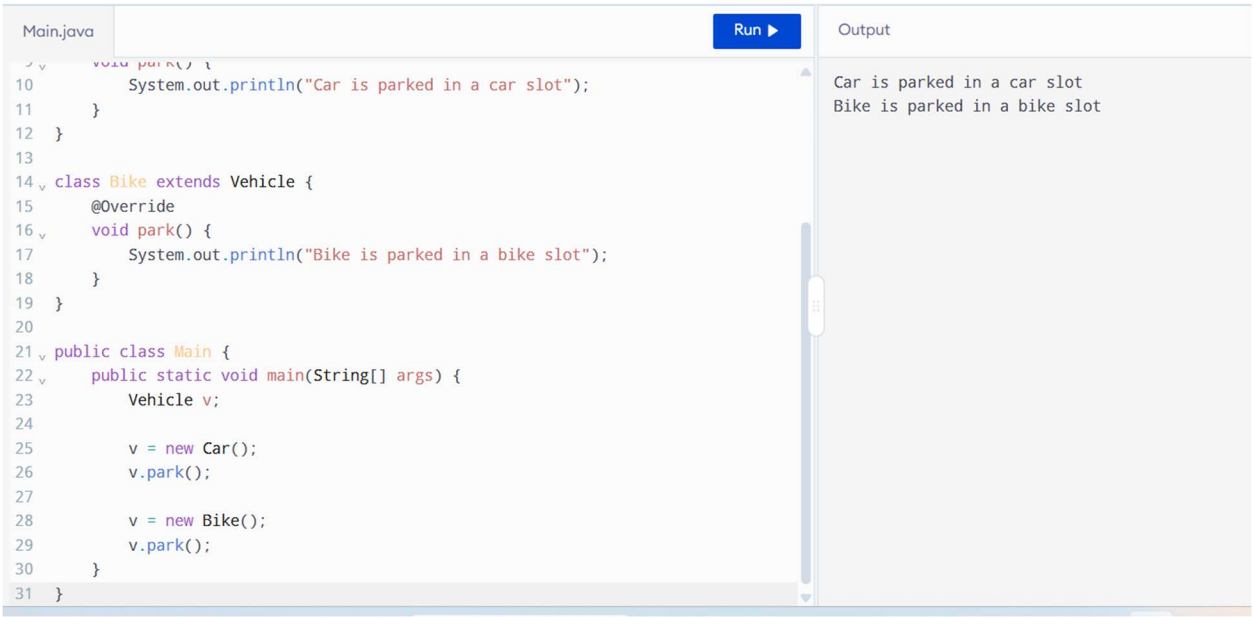
class Bike extends Vehicle {
    @Override
    void park() {
        System.out.println("Bike is parked in a bike slot");
    }
}

public class Main {
    public static void main(String[] args) {
        Vehicle v;
```

```
        v = new Car();
        v.park();

        v = new Bike();
        v.park();
    }
}
```

Output



Explanation

Polymorphism allows the same method name to perform different operations depending on the actual object. Here, the park() method is overridden in the Car and Bike classes.

Conclusion

Polymorphism makes the system flexible and easier to extend.

TASK 6: Use Interfaces

Aim

To define common parking operations using a Java interface.

Java Code

```
interface ParkingOperations {
    void park();
    void release();
}

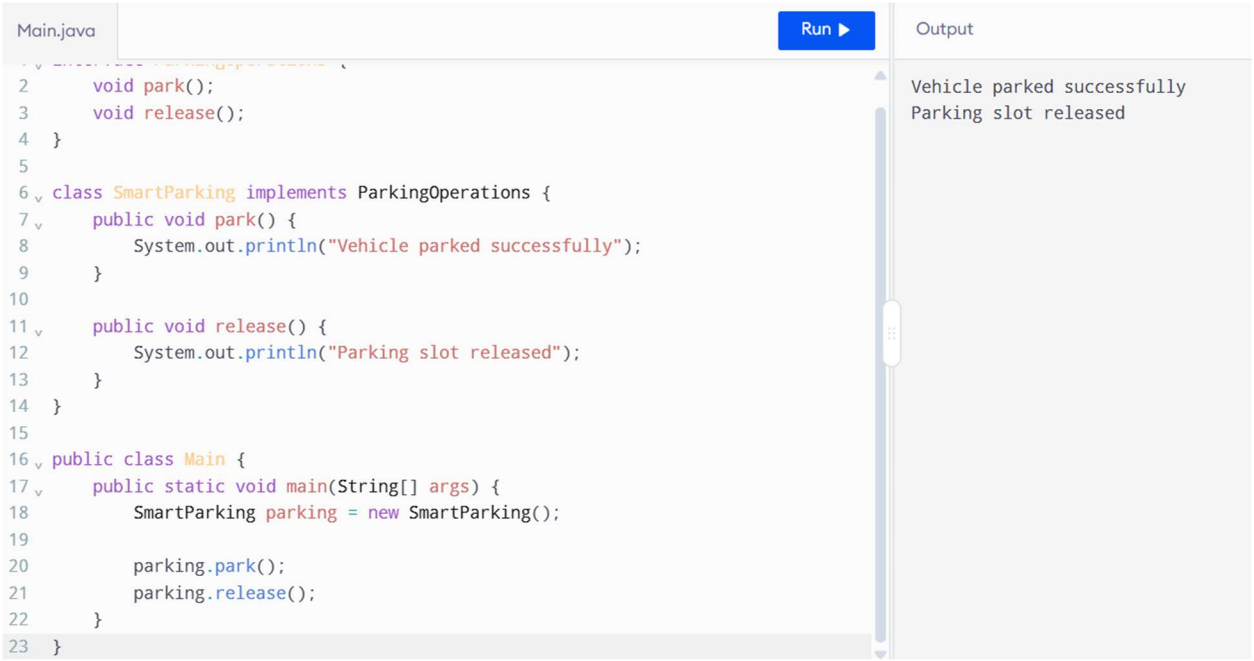
class SmartParking implements ParkingOperations {
    public void park() {
        System.out.println("Vehicle parked successfully");
    }

    public void release() {
        System.out.println("Parking slot released");
    }
}

public class Main {
    public static void main(String[] args) {
        SmartParking p = new SmartParking();

        p.park();
        p.release();
    }
}
```

Output



Explanation

An interface specifies methods that implementing classes must define. It supports abstraction and allows different classes to follow the same contract.

Conclusion

Interfaces provide a standard way to implement parking operations.

TASK 7: Handle Exceptions

Aim

To handle invalid parking operations using Java exception handling.

Java Code

```
public class Main {
    public static void main(String[] args) {
        int availableSlots = 0;


        try {
            if (availableSlots == 0) {
                throw new Exception("Parking is full");
            }


            System.out.println("Slot allocated");
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Output



Explanation

The try block contains code that may produce an error. The throw statement generates an exception, and the catch block handles it.

Conclusion

Exception handling prevents the program from terminating unexpectedly and provides meaningful error messages.

TASK 8: Complete the Parking Fee Calculation

Aim

To calculate parking fees based on the duration of parking.

Java Code

```
class ParkingFee {
    double calculateFee(int hours) {
        return hours * 20.0;
    }
}

public class Main {
    public static void main(String[] args) {
        ParkingFee fee = new ParkingFee();

        int hours = 3;
        double amount = fee.calculateFee(hours);

        System.out.println("Parking Hours: " + hours);
        System.out.println("Parking Fee: Rs. " + amount);
    }
}
```

Output

Main.java

Run ▶

```java
class ParkingFee {
    double calculateFee(int hours) {
        return hours * 20.0;
    }
}

public class Main {
    public static void main(String[] args) {
        ParkingFee fee = new ParkingFee();

        int hours = 3;
        double amount = fee.calculateFee(hours);

        System.out.println("Parking Hours: " + hours);
        System.out.println("Parking Fee: Rs. " + amount);
    }
}
```

Output

```
Parking Hours: 3
Parking Fee: Rs. 60.0
```

Explanation

The calculateFee() method multiplies the number of parking hours by ₹20 per hour. The calculated amount is returned to the calling method.

Conclusion

The fee calculation method makes parking charges easy to calculate and modify.

TASK 9: Use Collections to Manage Parking Slots

Aim

To store and display parking slots using the Java ArrayList collection.

Java Code

```
import java.util.ArrayList;

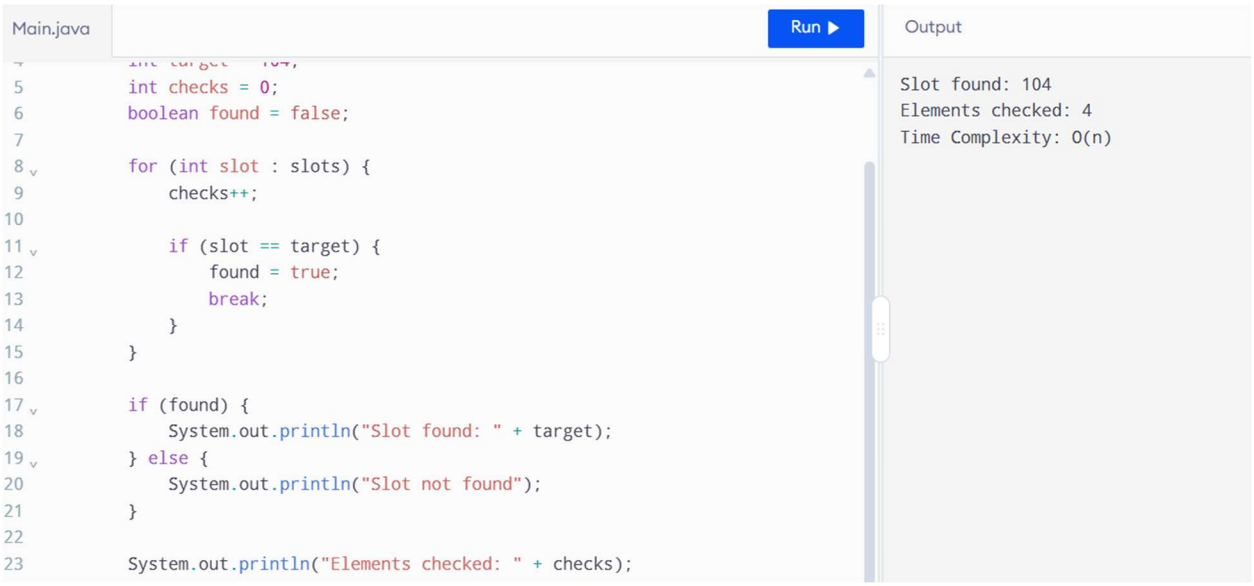

public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> slots = new ArrayList<>();


        slots.add(101);
        slots.add(102);
        slots.add(103);


        System.out.println("Available Parking Slots:");


        for (int slot : slots) {
            System.out.println(slot);
        }
    }
}
```

Output



Explanation

An ArrayList stores a dynamic collection of elements. The enhanced for loop visits each slot and displays its number.

Conclusion

Collections simplify the storage and management of parking slot information

TASK 10: Apply Big-O Complexity Analysis

Aim

To analyse the time complexity of common operations in the smart-parking system.

Operations and Complexity

Operation	Time Complexity
Access an element by array index	O(1)
Search for a slot using linear search	O(n)
Insert an element at the end of an ArrayList (amortized)	O(1)
Sort parking slot data	O(n log n)
Traverse all parking slots	O(n)

Explanation

Big-O notation describes how the running time of an algorithm grows as the input size increases.

For example, linear search checks parking slots one by one. In the worst case, it checks all n slots, so its time complexity is O(n).

Conclusion

Complexity analysis helps select efficient data structures and algorithms for managing large parking facilities.

TASK 11: Validate the System Using Test Cases

Aim

To validate the smart-parking system using basic test cases.

Test Cases

Test Case	Input	Expected Result
1	Available slot exists	Vehicle is parked
2	No slots available	Error message is displayed
3	Parking duration is 3 hours	Fee is ₹60
4	Release an occupied slot	Slot becomes available
5	Display slot numbers	All stored slot numbers are displayed

Explanation

Testing verifies whether the system behaves as expected for normal and exceptional conditions. Each operation should be tested using valid and invalid inputs.

Conclusion

Test cases help identify errors and improve the reliability of the smart-parking system.

TASK 12: Design Challenge – Extend the Smart-Parking System

Aim

To extend the campus smart-parking system to support different vehicle types, special pricing rules, visitor passes, and multiple floors.

Requirements

1. E-bikes: Charge a flat fee instead of an hourly fee.
2. Delivery vans: Charge twice the normal car rate between 9 AM and 5 PM.
3. Visitor passes: Allow visitors to park for one day using a day pass.
4. Multiple floors: Support five parking floors, each with a variable number of slots.

Proposed Design

The system can be extended using inheritance, polymorphism, encapsulation, and collections.

- A base Vehicle class stores common vehicle details.
- Car, EBike, and DeliveryVan classes represent different vehicle types.
- A ParkingFeeCalculator class calculates charges based on vehicle type and parking duration.
- A VisitorPass class stores pass details and validity.
- A ParkingFloor class manages the slots on each floor.
- A ParkingFacility class manages all five floors.

Sample Java Code

```
class Vehicle {
   String number;

   Vehicle(String number) {
      this.number = number;
   }

   double calculateFee(int hours) {
      return hours * 20.0;
```

```
    }
}

class EBike extends Vehicle {
    EBike(String number) {
        super(number);
    }

    @Override
    double calculateFee(int hours) {
        return 30.0;
    }
}

class DeliveryVan extends Vehicle {
    DeliveryVan(String number) {
        super(number);
    }

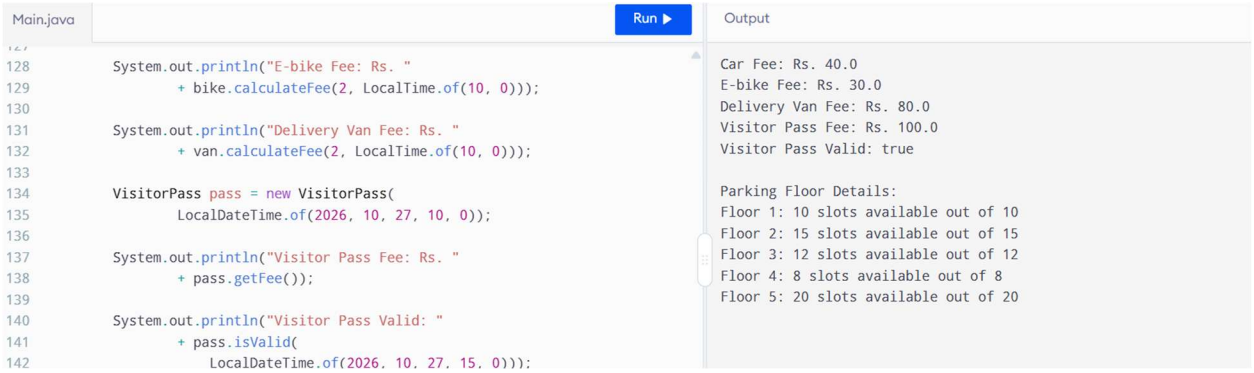
    @Override
    double calculateFee(int hours) {
        return hours * 40.0;
    }
}

public class Main {
    public static void main(String[] args) {
```

```
        Vehicle car = new Vehicle("CAR101");
        Vehicle bike = new EBike("EB101");
        Vehicle van = new DeliveryVan("VAN101");

        System.out.println("Car Fee: Rs. " +
                           car.calculateFee(2));
        System.out.println("E-bike Fee: Rs. " +
                           bike.calculateFee(2));
        System.out.println("Delivery Van Fee: Rs. " +
                           van.calculateFee(2));
    }
}
```

Output



Explanation

The program demonstrates runtime polymorphism. Each vehicle type calculates its fee differently by overriding the calculateFee() method.

For a complete implementation, the delivery van's special rate must be applied only during the specified 9 AM–5 PM period. The visitor pass should be valid for one day, and the facility should manage five floors with different slot capacities.

Conclusion

The extended design supports multiple vehicle categories, flexible pricing, visitor passes, and multi-floor parking management. Object-oriented programming makes the system modular, reusable, and easier to maintain.

Overall Conclusion

The campus smart-parking system demonstrates important Java object-oriented programming concepts, including classes, objects, encapsulation, abstraction, inheritance, polymorphism, interfaces, exception handling, and collections.

The design can be extended to support different vehicle types, dynamic parking fees, visitor passes, and multiple parking floors. Applying testing and Big-O complexity analysis helps improve the system's correctness and efficiency.

References

1. Herbert Schildt, Java: The Complete Reference.
2. Oracle Java Documentation: https://docs.oracle.com/en/java/
3. Java Collections Framework Documentation: https://docs.oracle.com/javase/8/docs/technotes/guides/collections/

GitHub Repository: https://github.com/karnatinikhil-sudo/CSA0922-Java-Programming